

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 08 · LECTURE NOTES

Articulated Bodies

Jointed systems, and the two ways to represent them.

MODULE OUTCOMES

Model common joints as constraints.

Compare maximal and reduced coordinate formulations.

Explain why long chains converge poorly and what to do about it.

Add joint limits and motors.

Build a ragdoll that behaves.

1 Joints are constraints

A free rigid body has six degrees of freedom: three translational and three rotational. A joint removes some of them, and **the number of scalar constraints a joint imposes equals the number of degrees of freedom it removes.**

Joint	DOF removed	DOF remaining	Constraint
Ball / spherical	3 translational	3 rotational	Two points, one fixed in each body, must coincide: $C = \mathbf{p}_a - \mathbf{p}_b = 0$.
Hinge / revolute	3 translational + 2 rotational	1 rotational	Ball joint, plus two axes must remain aligned.
Prismatic / slider	2 translational + 3 rotational	1 translational	Motion confined to a line; orientation fully locked.
Fixed / weld	all 6	none	Used for joints that may later break under load — λ from Module 04 gives you the force being transmitted, which is exactly the breaking criterion.
Universal	3 translational + 1 rotational	2 rotational	Two perpendicular hinges sharing a point.

NO NEW MACHINERY IS REQUIRED

Every joint is a constraint function $C(x) = 0$ of exactly the kind Module 04 introduced. The gradient gives the direction, the Lagrange multiplier gives the magnitude, and the same projection or impulse solver enforces it.

So joints, contacts, friction, and joint limits all feed one solver. A physics engine does not have a joint subsystem and a contact subsystem; it has a constraint solver, and these are different kinds of row.

This unification is the practical content of this module. Once you see it, articulated bodies stop being a separate topic and become an application of what you already have.

2 Maximal and reduced coordinates

	Maximal coordinates	Reduced coordinates
State	Every body carries its full 6 DOF. Joints are enforced as constraints between them.	Only the joint angles are state. Body positions are <i>derived</i> by walking the kinematic tree.
Simplicity	Simple and uniform. Free bodies, jointed bodies, and contacts all go through the same path.	More complex. The kinematic tree, its traversal, and the Jacobians all have to be built.
Joint exactness	Approximate. The solver satisfies the constraint to within its tolerance, so joints visibly separate under load.	Exact. A broken joint is not representable, because the coordinates cannot express one.
Variable count	6 per body, plus constraints.	1–3 per joint. Far fewer.
Closed loops	Natural — just another constraint.	Awkward — a loop breaks the tree structure and requires additional constraints anyway.
Contact	Natural — contacts are constraints like any other.	Awkward — contact acts on bodies, which are not the state variables.
Used by	Game engines. Bullet, PhysX, Havok, Box2D.	Robotics. MuJoCo, Drake, and control applications generally.

FEATHERSTONE'S ALGORITHM

The direct reduced-coordinate approach requires forming and inverting the joint-space mass matrix, which is dense and gives an $O(n^3)$ cost in the number of joints — prohibitive for a detailed character.

Featherstone's articulated body algorithm computes the same answer in $O(n)$ by propagating *articulated inertias* along the kinematic tree: an inward pass accumulating the effective inertia each link presents to its parent, then an outward pass computing accelerations.

It is exact rather than iterative. Joints are satisfied perfectly at every step, at any step size, with no drift and no convergence parameter.

Every serious robotics simulator is built on it, and Bullet offers it alongside its maximal-coordinate path. It is genuinely complicated to implement, and the payoff — exactness and linear cost simultaneously — is why it is worth knowing exists even if you never write one.

3 Chains

INFORMATION TRAVELS ONE LINK PER ITERATION

Gauss–Seidel satisfies constraints one at a time in sequence. When a correction is applied at one constraint, the neighbouring constraint sees it on the next visit — and the one after that on the visit after.

So a disturbance at one end of a chain reaches the other end only after as many iterations as there are links. A 100-link rope solved with 10 iterations behaves as though the far end is unaware the anchor exists; the chain stretches visibly near the top, where the load is greatest.

This is not a bug, and raising the iteration count is not a solution. Full convergence needs iterations proportional to chain length, and each iteration costs time proportional to chain length too — so the cost grows as the square.

It is also why PBD ropes look slightly elastic however the stiffness is set, and why rigid chains are one of the places reduced coordinates genuinely win.

- **Order the constraints along the chain**, from root to tip. A single forward pass then propagates information the entire length, because each constraint is visited after the one before it has already been corrected. **This is nearly free and makes the largest single difference.**
- **Alternate the direction** each iteration — forward, then backward. A purely forward sweep biases the solution toward the tip; alternating removes the asymmetry.
- **Increase mass toward the root.** A heavier anchor absorbs proportionally less correction, which is both physically reasonable for most real chains and helpful for convergence.
- **Solve hierarchically.** Treat the chain coarsely first — every fourth link — then refine. This is the multigrid idea, and it converges in $O(\log n)$ passes rather than $O(n)$.
- **Substep.** As in every other module, smaller steps with fewer iterations outperform larger steps with more.

4 Ragdolls

Ingredient	Why it matters
Joint limits	An elbow that bends backwards is noticed instantly , even by someone who could not say what is wrong. The cheapest single improvement.
Realistic mass ratios	A human head is about 8% of body mass, a forearm about 2%. Uniform masses produce motion that reads as subtly wrong — limbs swing with the wrong period.
Joint damping	Real joints resist motion through soft tissue. An undamped ragdoll flails in a way that reads as weightless rather than limp.
Self-collision	An arm passing through the torso destroys the illusion completely. Expensive, and necessary.

Ingredient	Why it matters
Blending with animation	A pure ragdoll looks dead , because a falling person is not passive — they brace, reach, and tense. Production systems blend from animation into ragdoll on impact and sometimes back out again, which is why game ragdolls often look better than physically more faithful ones.

JOINT LIMITS ARE INEQUALITY CONSTRAINTS, SO THEY ARE CONTACTS

A knee bends from roughly 0° to 150° . Expressed as constraints: $C = \theta \geq 0$ and $C = 150^\circ - \theta \geq 0$. Two **inequality** constraints.

Inequality constraints are exactly what contact is (Module 06), so they go into the same solver with the same treatment: active only when violated, impulse clamped to be non-negative, a small slop to prevent oscillation at the boundary.

Make them slightly soft. A perfectly hard limit stops the joint dead, which reads as mechanical — a hinge rather than a body. A small compliance (Module 04's α) gives a short deceleration that reads as flesh and ligament.

This is the single cheapest improvement available to a ragdoll, and it is routinely skipped in first implementations because the ragdoll 'works' without it.

5 Motors

- **A motor is a constraint with a target and a force limit:** drive this joint toward a desired angle, or a desired angular velocity, using no more than this much torque.
- **The force limit is the important part.** An unlimited motor is effectively a teleporter — it will achieve its target regardless of what is in the way, fighting contacts and other constraints until the solver fails or something is flung across the scene.
- **PD control** — torque proportional to angle error plus a term proportional to angular velocity error — is the standard driver. **Tuning the two gains is most of the work**, and nobody gets it right on the first attempt.
- **This is how physically-driven characters work:** an animation supplies target joint angles frame by frame, and motors attempt to achieve them while gravity, contact, and momentum act on the body. The character responds to being pushed because the motors are not infinitely strong.
- **And this is how they fail.** Gains too low and the character sags and collapses under its own weight; too high and it becomes a rigid armature that ignores the world and jitters against it. The usable range is narrower than it looks.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Featherstone — Rigid Body Dynamics Algorithms	The reduced-coordinate algorithm of §2, from its author. His site has free tutorial material and spatial-vector notes.
Erin Catto — joints and motors (GDC, free)	The maximal-coordinate side: joint constraints, limits, and motors as implemented in a production engine.
Bullet Physics manual and source	Both formulations in one codebase — compare <code>btGeneric6DofConstraint</code> against the Featherstone path.
Ten Minute Physics — chains and ragdoll episodes	The chain convergence problem of §3, demonstrated interactively.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement ball and hinge joints as constraints in your Module 04 solver. Verify the constrained degrees of freedom are actually removed.
- 2 Build a chain of 100 links. Plot the stretch along its length for 1, 10, and 100 solver iterations.
- 3 Reorder the constraints along the chain and repeat. Report the improvement.
- 4 Implement alternating-direction sweeps and measure the difference against a single-direction sweep.
- 5 Implement a hierarchical chain solve and compare convergence against the flat solver at equal cost.
- 6 Build a ragdoll with realistic mass ratios, taken from published anthropometric data. Compare against uniform masses.
- 7 Add joint limits as inequality constraints. Compare a hard limit against a compliant one and describe the difference in how the motion reads.
- 8 Implement a PD motor on a single joint and tune the gains. Document the gain values at which the joint sags and at which it becomes rigid.
- 9 Drive a ragdoll's joints toward an animated pose with limited motors. Push it and observe the recovery.
- 10 Write a page comparing maximal and reduced coordinates for a specific application of your choosing, and justify a recommendation.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How many constraints does a joint impose, and how do you know?
- 2 Give the constraint functions for a ball joint and a hinge.

- 3 Compare maximal and reduced coordinates on five axes.
- 4 What does Featherstone's algorithm achieve, and why does it matter?
- 5 Why do long chains converge poorly in an iterative solver, and why does raising the iteration count not solve it?
- 6 Give four mitigations for the chain problem.
- 7 Name five ingredients of a convincing ragdoll.
- 8 Why are joint limits inequality constraints, and why make them soft?
- 9 What is a motor, why does the force limit matter, and how do PD gains fail in each direction?